

Mastermind + Wordle — AI Usage

Project 2, Phase II · EID ak54582

Updated from the Phase I AI.pdf to cover the Wordle extension

1. AI tools used

Tool	Used for
Claude Code (Anthropic), model Claude Opus 5.5, run as the VS Code extension	Phase II: checking which Phase I version to start from; implementing the Wordle extension and the refactoring of my Phase I code; writing the Wordle tests and the Mastermind regression tests; creating the word list <code>data/words.txt</code> ; compiling and running the tests (including on Java 8) and a planted-bug check; drawing the before/after diagrams; drafting ARCHITECTURE_CHANGE, TESTS and this document.

What changed from Phase I. In Phase I no AI tool wrote submitted code; the AI only reviewed, tested and documented. In Phase II **the AI wrote most of the code changes:** the new Wordle types, the shared `SecretSource` / `FixedSecretSource` / `PositionMatcher` types, the changes to `AbstractGuessingGame` , `MastermindGame` , `MastermindScorer` and `Driver` , and the new tests. My own Phase I code carries over: `ConsoleRunner` , `GuessingGame` and `Turn` are unchanged, and the Mastermind rules, messages and the original 80-check test suite are my Phase I work. ARCHITECTURE_CHANGE.md has a "Who decided what" table that marks each design decision as mine (from Phase I) or an AI suggestion I accepted.

2. Important AI interactions

Interaction 1: Making sure Phase II starts from what I submitted

What I asked	Verify that <code>Project2_Phase1_ak54582.zip</code> is my previous assignment, then build Phase II on top of it.
What the AI did	Found two different Phase I versions on my machine: the zip (dated Sep 22) and an older folder (Sep 21) with extra stress-testing changes and 119 tests. It compared dates and diffs and noted that the zip is newer, and that its AI.pdf says the code is mine and that I rejected the AI's suggested parsing change. It concluded the zip was what I submitted, built Phase II on it, and did not bring in the changes from the unsubmitted folder.
Why it mattered	The assignment requires starting from the submitted Phase I codebase. Quietly building on the wrong version would have mixed unsubmitted AI changes into the "before" state of the report.

Interaction 2: Planning the refactor from my Phase I design

What I asked	Extend the design to support Wordle responsibly, without duplicating the game runner or console code.
What the AI did	Read my Phase I Design.pdf and AI.pdf first. It carried out the refactor I had planned in Phase I (a game-independent secret source in place of <code>CodeGenerator</code>) and proposed two further changes: moving the secret's life cycle into <code>AbstractGuessingGame</code> , and pulling the repeated-symbol rule out of <code>MastermindScorer</code> into a shared <code>PositionMatcher</code> . The second one reverses a position I took in Phase I, and the AI pointed that out explicitly. It left my three core types (<code>GuessingGame</code> , <code>ConsoleRunner</code> , <code>Turn</code>) untouched and, like me in Phase I, decided against generic types.

How it was checked	Wordle's specification states the same rule as Mastermind ("CORRECT matches are assigned before PRESENT; a secret letter occurrence cannot satisfy more than one guessed letter"), which is what justifies sharing one implementation. The Phase I exhaustive tests, which compare every Mastermind secret with every guess against an independent scorer, still pass on the shared code.
--------------------	---

Interaction 3: Proving Mastermind did not change

What the AI did	Before editing anything, it compiled my Phase I zip, ran its 80 tests (80/80), and captured the complete console output of two scripted Mastermind sessions from that build. Those transcripts became regression tests: the Phase II output must match them character for character.
Result	Both transcripts match exactly. The Phase I solver statistics (all 1,296 secrets won, worst case 9 guesses on RYRP, average 5.765) are also identical, as expected if scoring behaves the same.

Interaction 4: Where the AI got things wrong, and how testing caught it

Encoding bug	The AI meant to write Java escapes such as <code>\u00c4</code> and <code>\uffeff</code> , but the files ended up containing the literal characters. On Windows, <code>javac</code> reads source files as Cp1252 by default, so a test for a word file with a byte-order mark failed, and some fuzz inputs were silently different from what the test claimed. Fixed by converting every <code>.java</code> file to pure ASCII and re-running everything.
Wrong expected value	One expected feedback was worked out by hand incorrectly (<code>P A P A P</code> for secret CRANE, guess ALLEY; the correct answer is <code>P A A P A</code>). It was caught by re-deriving the value before the first run.
Weak test	The planted-bug check showed that one Wordle test <i>crashed</i> on a broken implementation instead of reporting a failure. It was rewritten so the failure is reported. One planted bug was itself written badly (it threw an exception, so its failure counts were incomplete) and was redone.
Hang	Once, while the planted-bug script ran under Python, a test run hung for 10 minutes; it could not be reproduced. As a safeguard, the helper that launches the real program now kills a child process after 60 seconds, so a hang fails one check instead of freezing the suite.
Lesson	As in Phase I, the AI was fast and thorough but not reliable on its own; every one of these problems was found by running and checking, not by reading the code.

Interaction 5: Documents and PDFs

The same approach as Phase I: HTML printed to PDF with headless Microsoft Edge. ARCHITECTURE_CHANGE.md is the single source for ARCHITECTURE_CHANGE.pdf, so the two cannot drift apart. The class diagrams are generated by a script and were checked by rendering them to images, which exposed overlapping labels and text overflowing boxes; those were fixed. Sources are in `docs/source/` (`python build_diagrams.py`, then `python build_pdfs.py`).

3. AI suggestions I accepted

The full list with reasons is in ARCHITECTURE_CHANGE.md (decisions D6–D13 and D15). In short:

- A non-generic `SecretSource`, with secrets checked by the game rather than by the source (D6).
- The secret's life cycle in `AbstractGuessingGame`, with the rule "a secret must be a legal guess" (D7).
- The shared `PositionMatcher` used by both scorers (D8).
- Renaming `GameConfiguration` and `Feedback` to Mastermind-specific names (D9).
- Leaving `ConsoleRunner`, `GuessingGame` and `Turn` unchanged, with both games printing `Result: ...` (D10).
- One testing-mode convention for both games, `-D<game>.secret`, checked at startup (D11).
- The word-list file format and lookup (D12), and no generic types (D13).
- Golden-transcript regression tests for Mastermind (D15).

4. Suggestions not taken

A Phase I decision of mine that still stands

In Phase I I rejected the AI's suggestion to make numeric `-D` settings strict, so that, for example, `-Dmastermind.guesses=abc` is an error instead of quietly using the default (my reasons are in the Phase I AI.pdf). Phase II keeps that decision and applies it consistently: the new `-Dwordle.guesses` and `-Dwordle.length` settings behave the same way as Mastermind's (D14).

Alternatives the AI considered and rejected itself

- **Generic types** (`GuessingGame<F>`, `Turn<F>`, `SecretSource<S>`). Both games use strings for secrets and guesses, and the runner only prints feedback, so type parameters would add noise everywhere for no benefit.
- **A game registry or plugin system** in `Driver`. With two games, a `switch` is clearer.
- **A shared configuration base class**. The only shared setting is the attempt limit, which the base class already receives as an `int`.
- **Printing Feedback: instead of Result:** to match the assignment's example. It would have changed Mastermind's Phase I output for no functional gain.
- **A separate list of possible answers** in addition to the allowed guesses. The specification works with one list, and a second file would add configuration without changing the architecture.

5. How the result was checked

- 220 checks (129 Wordle, 91 Mastermind) pass when compiled with JDK 17 (`--release 8`) and on a real JDK 8.
- All 80 Phase I checks pass unchanged, and two Mastermind sessions match the Phase I build character for character.
- Six deliberately planted bugs were each caught by the suites (TESTS.pdf).
- Both games were also played by hand through `java assignment2.Driver wordle test` and `mastermind test`, including launching from inside `out/` and the error cases (missing word file, bad test secret).